

Code generation

GitHub Repository: https://github.com/simonhoumb/tdt4242_exercise2

Screenshots of successful execution

Starting the backend server:

```
C:\Dev\NTNU\tdt4242_advanced_software\tdt4242_exercise2 [main = +1 ~9 -0 !]> npm run dev --workspace backend
> aiguidebook-backend@1.0.0 dev
> node --watch src/server.js

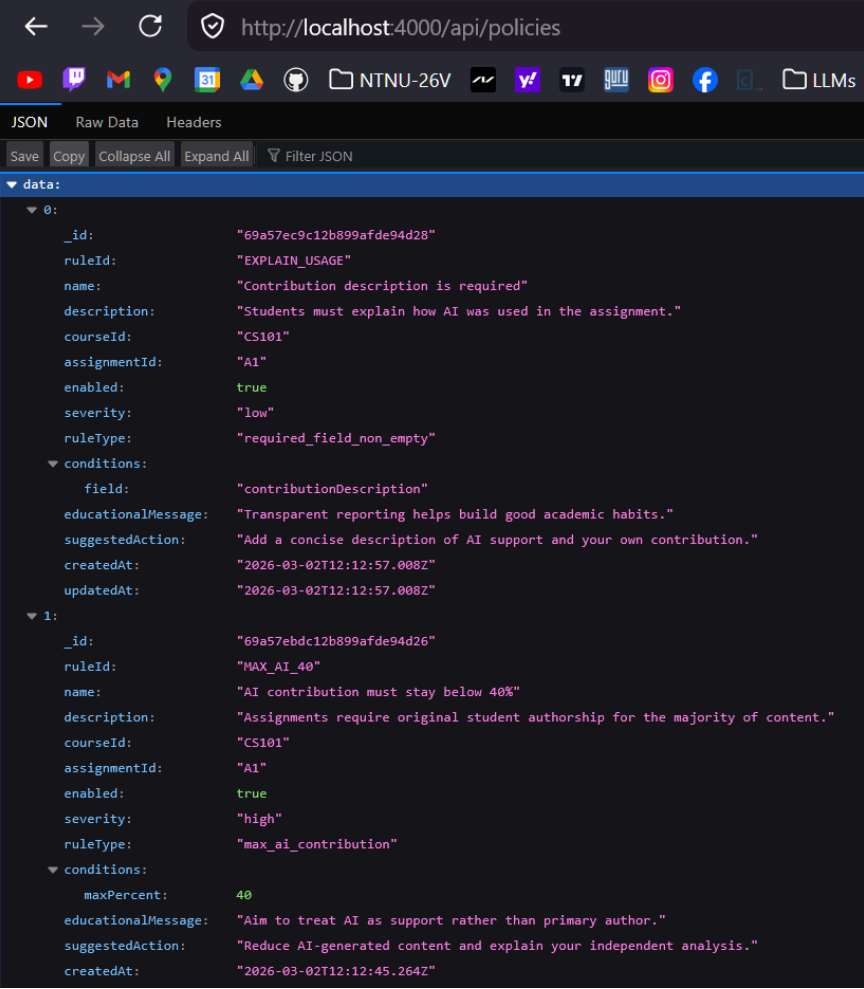
Backend listening on http://localhost:4000
```

Starting the frontend server:

```
C:\Dev\NTNU\tdt4242_advanced_software\tdt4242_exercise2 [main = +1 ~9 -0 !]> npm run dev --workspace frontend
VITE v5.4.21 ready in 2336 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Example policies seen through the backend API:



The screenshot shows a web browser window with the address bar displaying `http://localhost4000/api/policies`. The browser's developer tools are open, showing the JSON response from the API. The response is a list of two policy objects under the `data` key.

```
JSON
Raw Data
Headers
Save Copy Collapse All Expand All Filter JSON
```

```
{
  "data": [
    {
      "_id": "69a57ec9c12b899afde94d28",
      "ruleId": "EXPLAIN_USAGE",
      "name": "Contribution description is required",
      "description": "Students must explain how AI was used in the assignment.",
      "courseId": "CS101",
      "assignmentId": "A1",
      "enabled": true,
      "severity": "low",
      "ruleType": "required_field_non_empty",
      "conditions": {
        "field": "contributionDescription",
        "educationalMessage": "Transparent reporting helps build good academic habits.",
        "suggestedAction": "Add a concise description of AI support and your own contribution."
      },
      "createdAt": "2026-03-02T12:12:57.008Z",
      "updatedAt": "2026-03-02T12:12:57.008Z"
    },
    {
      "_id": "69a57ebdc12b899afde94d26",
      "ruleId": "MAX_AI_40",
      "name": "AI contribution must stay below 40%",
      "description": "Assignments require original student authorship for the majority of content.",
      "courseId": "CS101",
      "assignmentId": "A1",
      "enabled": true,
      "severity": "high",
      "ruleType": "max_ai_contribution",
      "conditions": {
        "maxPercent": 40,
        "educationalMessage": "Aim to treat AI as support rather than primary author.",
        "suggestedAction": "Reduce AI-generated content and explain your independent analysis."
      },
      "createdAt": "2026-03-02T12:12:45.264Z"
    }
  ]
}
```

Example of manual logging that does not comply with the policy:

Ethical AI Usage Guidance

Use AI as a thinking partner, not a replacement for your own analysis.

- Document what AI changed in your work.
- Preserve your own reasoning and decision-making process.
- Use AI for feedback, clarity, and iteration instead of full answer generation.

Document AI Usage

Student ID

student1

Assignment ID

A1

Assistance Type

editing

How AI contributed

AI was used for spelling, grammar, and sentence improvement

Course ID

CS101

AI Tool Name

ChatGPT

Estimated AI Contribution (%)

55

Prompt excerpt (optional)

Can you please improve this paragraph:
this is an example paragraph, test123

Save usage log

Compliance Feedback

Status: Needs attention

Great effort documenting your AI usage. Keep strengthening your independent thinking by applying the feedback below.

AI contribution must stay below 40%

Rule: MAX_AI_40

Assignments require original student authorship for the majority of content. Triggered by: 55%.

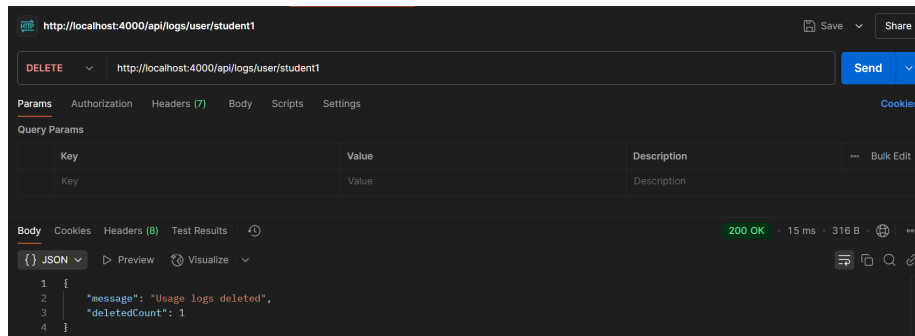
Action: Reduce AI-generated content and explain your independent analysis.

Severity: high

Recent Usage Logs

Total logs: 0

Usage logs can be deleted by sending a DELETE request with the specific userID. Here it is visualized by using PostMan.:



The web application does not support doing this through the user interface, as it was not specifically stated in the requirements. It is only stated that it should be possible to do it using an endpoint. This could however be implemented at a later stage

Explanation of the role of each major component

The project is a full-stack application. It uses Express for the server and React for frontend. MongoDB is used to persist data locally.

The code is divided into two folders, *backend* and *frontend*.

Backend

backend/src/controllers/

Implementations of the endpoints of the application. There are two controllers: One for the *log* entity, and one for the *policy* entity. Includes CRUD operations for these entities, as well as some specific operations like checking compliance.

backend/src/models/

Contains MongoDB data models schemas (using Mongoose), describing how the entities are stored. The two models are *PolicyRule* and *UsageLog*.

backend/routes/

Defines the routes of the application. This handles the log and compliance CRUD operations of the API.

backend/services/

Contains the “business logic” of our application. Includes a compliance service that finds any violations based on defined policies and gives educational feedback. Also a logging service that provides functions for saving usage logging.

backend/utils/

Contains common utility functions used throughout the application. Includes an error handler and validator. Used by the controllers.

backend/app.js

Sets up and configures the express web server. This includes adding middleware and routes.

backend/server.js

“Entry point” of the backend. Creates the app (using app.js), starts the server and connects to the database.

Frontend**frontend/src/App.jsx**

Main page component of the web application.

frontend/src/features/usage-compliance/components

Contains components showing compliance feedback, proactive tips from the ethical guidance card, and usage log form where the user fills in the information of their AI usage.

frontend/src/features/usage-compliance/hooks/

Manages the API calls for usage logs and compliance checks.

AI generated code vs. manual

All code was generated using the GitHub Copilot extension in VSCode. Here we instructed the AI agent by using the prompting strategy from exercise 2.1. We found some minor bugs, but these were easily corrected by prompting.

We have however removed some code that we found useless. Copilot created a file called “autoLoggingMiddleware”. The intention of this middleware was to intercept and capture AI usage of the user (requests to various AI tools). It did not manage to do this (perhaps not surprising). We consider this a very hard, if not impossible, thing to implement. In a real-world scenario, this would have to be further discussed with stakeholders. For our purposes, we have simply removed the middleware and not implemented the two requirements that describe this (FR4 and NFR4).